



JMPM TECNOLOGIA

TECNOLOGIA & DESENVOLVIMENTO

MANUAL DO USUÁRIO

Motor Core GraphQL

Guia de utilização da API GraphQL exposta pelo Motor Core GraphQL para Protheus

Versão **3.1.0**

JMPM Tecnologia

05/09/2026

Sumário

Como Começar

O que é este projeto

Pré-requisitos

Passo 1 — Compilar e implantar

Passo 2 — Testar a introspecção

Passo 3 — Ver os campos de uma tabela

Passo 4 — Fazer sua primeira consulta

Passo 5 — Fazer sua primeira mutation (opcional)

Onde ir a partir daqui

Manual de Utilização

Endpoint

Introspecção

Consultas (queries)

Mutations (escrita)

Formato de erros

Coisas que a API intencionalmente não faz (ainda)

Manual do Usuário — Este manual orienta usuários e desenvolvedores no uso do Motor Core GraphQL para Protheus: como colocar o serviço no ar, executar consultas, mutations e introspecção, e interpretar erros e limitações da API.

Como Começar

Guia rápido para colocar o motor GraphQL do Protheus em funcionamento e executar sua primeira consulta em poucos minutos.

O que é este projeto

Um servidor GraphQL nativo em TLPP que expõe o dicionário de dados do Protheus (tabelas SX2/SX3, relacionamentos SX9) como um schema GraphQL **dinâmico** — nenhum tipo é escrito à mão. Toda tabela liberada aparece automaticamente como um tipo consultável, com seus campos reais.

Hoje o motor cobre: - **Leitura**: consultas paginadas e filtráveis sobre qualquer tabela não bloqueada, incluindo campos de relacionamento aninhados (via SX9). - **Escrita**: mutations `create / update / delete` sobre tabelas explicitamente liberadas.

Pré-requisitos

- Um AppServer Protheus já rodando, com REST habilitado (`[HTTPREST]` no `appserver.ini`) na porta configurada (`9995` nos exemplos abaixo).
- Acesso para compilar e implantar fontes `.tlpp` nesse ambiente.

Passo 1 — Compilar e implantar

Compile todos os fontes em `custom/backoffice/graphql/core/` e `custom/backoffice/graphql/entrypoints/` e implante o RPO resultante no seu AppServer, seguindo o processo de build já usado no seu ambiente.

Atenção: o arquivo de configuração `custom/backoffice/graphql/config/graphql-config.json` **não** faz parte do RPO compilado — ele precisa ser copiado manualmente para o `RootPath` do AppServer (veja `appserver.ini`, seção `[P12] RootPath=`), não para o `SourcePath`. Sem esse arquivo no lugar certo, a lista de bloqueio fica vazia silenciosamente e **todas** as tabelas ficam visíveis — confirmado em ambiente real durante o desenvolvimento. Detalhes completos no `manual-implementacao.md`.

Passo 2 — Testar a introspecção

Com o serviço no ar, confirme que o endpoint responde:

```
curl "http://localhost:9995/rest/graphql"
```

A resposta lista os nomes de todas as tabelas liberadas (respeitando a lista de bloqueio do `graphql-config.json`).

Passo 3 — Ver os campos de uma tabela

```
curl "http://localhost:9995/rest/graphql?type=SA1"
```

Retorna todos os campos reais de `SA1` (nome, tipo GraphQL, tamanho máximo, se é obrigatório), extraídos ao vivo do SX3.

Passo 4 — Fazer sua primeira consulta

```
{ SA1(limit: 5, filter: [{field: "A1_COD", op: "eq", value: "000001"}]) {
  A1_COD
  A1_NOME
  SC5 { C5_NUM }
} }
```

Via `curl` (texto do GraphQL precisa ir codificado na URL):

```
curl "http://localhost:9995/rest/graphql?
query=%7B%20SA1(limit%3A%205)%20%7B%20A1_COD%20A1_NOME%20%7D%20%7D"
```

Passo 5 — Fazer sua primeira mutation (opcional)

Mutations só funcionam em tabelas explicitamente liberadas em `allowMutations` no `graphql-config.json` (nenhuma por padrão). Supondo que `SA1` esteja liberada:

```
mutation { createSA1(input: {A1_COD: "000123", A1_LOJA: "01", A1_NOME: "Foo"}) {
  A1_COD
  A1_NOME
} }
```

Atenção: mutations exigem a palavra-chave `mutation` no início do texto — sem ela, o servidor interpreta como uma consulta de leitura comum e retorna erro de "tabela desconhecida".

Onde ir a partir daqui

- `manual-implementacao.md` — como implantar, configurar e operar este motor em um ambiente Protheus real (passo a passo completo, incluindo a pegadinha do `RootPath` e as limitações conhecidas).
- `manual-utilizacao.md` — referência completa da linguagem GraphQL suportada: sintaxe de consultas, filtros, paginação, relacionamentos e mutations, com exemplos.
- `architecture.md` — arquitetura interna do motor, para quem for estender ou depurar o código.

Manual de Utilização

Referência completa da API GraphQL exposta por este motor. Público-alvo: desenvolvedores que vão consumir o serviço a partir de outro sistema, integração ou aplicação.

Endpoint

Todo o acesso é feito por um único endpoint REST:

```
GET /rest/graphql
```

Uso	Exemplo
Listar tipos disponíveis (introspecção)	<code>GET /rest/graphql</code>
Ver os campos de uma tabela	<code>GET /rest/graphql?type=SA1</code>
Executar uma consulta ou mutation	<code>GET /rest/graphql?query=<texto GraphQL codificado na URL></code>

O texto GraphQL vai sempre no parâmetro `query`, url-encoded. Os exemplos abaixo mostram o texto GraphQL "cru" — para usá-lo via `curl`, codifique-o (`urllib.parse.quote_plus` em Python, ou qualquer codificador de query string).

Introspecção

Listar tabelas disponíveis

```
curl "http://localhost:9995/rest/graphql"
```

```
{"data":{"__schema":{"queryType":{"name":"Query"},"types":[{"name":"SA1"}, {"name":"SA2"}, ...]}}}
```

Só aparecem tabelas **não** bloqueadas pela configuração do servidor — tabelas bloqueadas nunca aparecem aqui, independentemente de você saber o nome delas.

Ver os campos de uma tabela

```
curl "http://localhost:9995/rest/graphql?type=SA1"
```

```
{
  "data": {
    "__type": {
      "name": "SA1",
      "fields": [
        {
          "name": "A1_COD",
          "sx3Type": "C",
          "graphqlType": "String",
          "maxLength": 6,
          "required": false
        },
        {
          "name": "A1_NOME",
          "sx3Type": "C",
          "graphqlType": "String",
          "maxLength": 50,
          "required": false
        },
        ...
      ],
      "relations": [
        {
          "name": "SC5",
          "type": "[SC5]",
          "cardinality": "MANY"
        }
      ]
    }
  }
}
```

Cada campo traz: nome real, tipo AdvPL/SX3 original, tipo GraphQL mapeado, tamanho máximo e se é obrigatório. Campos bloqueados por configuração ou marcados como não-visíveis no dicionário (`X3_VISUAL = "N"`) nunca aparecem aqui.

Consultas (queries)

Sintaxe básica

```
{ SA1(limit: 5) {
  A1_COD
  A1_NOME
} }
```

Retorna até 5 registros de `SA1` , trazendo apenas os campos pedidos.

Paginação

Toda listagem aceita `limit` e `offset` :

```
{ SA1(limit: 20, offset: 40) {
  A1_COD
} }
```

- `limit` — sem informar, usa o padrão configurado (`defaultPageSize`). Sempre limitado ao teto do servidor (`maxPageSize`), mesmo que você peça mais.
- `offset` — sem informar, começa do início (`0`).

Filtros

```
{ SA1(filter: [{field: "A1_COD", op: "eq", value: "000001"}]) {
  A1_COD
  A1_NOME
} }
```

Operadores suportados: `eq` (igual), `gt` (maior que), `gte` (maior ou igual), `lt` (menor que), `lte` (menor ou igual). Vários filtros no mesmo array são combinados com `AND` :

```
{ SA1(filter: [
  {field: "A1_COD", op: "gte", value: "000100"},
  {field: "A1_COD", op: "lt", value: "000200"}
]) {
  A1_COD
} }
```

Todo valor de filtro é escapado automaticamente antes de chegar ao banco — não há risco de injeção de SQL via filtro.

Campos de relacionamento (aninhamento)

Relacionamentos declarados no dicionário do Protheus (SX9) aparecem como campos aninhados, resolvidos automaticamente:

```
{ SA1(limit: 5) {
  A1_COD
  A1_NOME
  SC5 { C5_NUM C5_EMISSAO }
} }
```

Isso traz, para cada cliente, os pedidos de venda relacionados (via a regra de vínculo do SX9 entre SA1 e SC5). Relacionamentos podem ser 1:1 ou 1:N (lista) — o tipo de retorno já reflete isso na introspecção ("type": "[SC5]" para 1:N, "type": "SC5" para 1:1). Não há limite de profundidade de aninhamento imposto pelo servidor.

Apelidos (alias)

Para pedir a mesma tabela mais de uma vez na mesma consulta, com critérios diferentes, use apelidos:

```
{
  recentes: SA1(limit: 5, filter: [{field: "A1_COD", op: "gte", value: "000900"}])
  { A1_COD }
  antigos: SA1(limit: 5, filter: [{field: "A1_COD", op: "lt", value: "000100"}])
  { A1_COD }
}
```

A resposta traz cada resultado sob a chave do apelido, não do nome da tabela.

Mutations (escrita)

Mutations só existem para tabelas explicitamente liberadas pelo administrador do servidor (veja `manual-implementacao.md`). Tentar uma mutation em tabela não liberada retorna erro claro, não um erro de servidor.

Toda mutation precisa começar com a palavra-chave `mutation` — sem ela, o texto é interpretado como consulta de leitura comum.

Criar um registro

```
mutation { createSA1(input: {A1_COD: "000123", A1_LOJA: "01", A1_NOME: "Cliente Novo"}) {
  A1_COD
  A1_NOME
} }
```

O campo de filial nunca precisa (e não deve) ser enviado em `create` — o servidor sempre usa a filial da sessão atual, ignorando qualquer valor enviado pelo cliente nesse campo.

A resposta traz o registro recém-criado, moldado exatamente pelos campos pedidos na seleção — igual a uma consulta normal.

Atualizar um registro

```
mutation { updateSA1(input: {A1_FILIAL: "01", A1_COD: "000123", A1_LOJA: "01", A1_NOME:
"Nome Atualizado"}) {
  A1_NOME
} }
```

Update é sempre parcial: envie no `input` os campos de chave (obrigatórios, para identificar a linha) mais **somente** os campos que você quer alterar. Campos omitidos permanecem com o valor atual, inalterados.

Os campos de chave dependem do índice de ordem 1 real da tabela no Protheus — para `SA1`, são `A1_FILIAL`, `A1_COD` e `A1_LOJA`. Se algum campo de chave estiver faltando no `input`, a mutation retorna erro explícito (`"Missing key field '<campo>' for update"`) em vez de adivinhar.

Se a chave não corresponder a nenhum registro ativo, a resposta é:

```
{"errors":[{"message":"Row not found for update"}]}
```

Excluir um registro

```
mutation { deleteSA1(input: {A1_FILIAL: "01", A1_COD: "000123", A1_LOJA: "01"}) {
  A1_COD
} }
```

A exclusão é sempre lógica (soft-delete) — o registro deixa de aparecer em consultas normais, mas o dado físico permanece, exatamente como qualquer outra exclusão no Protheus. A resposta traz os dados do registro **antes** da exclusão, moldados pela seleção pedida.

Assim como em `update`, uma chave que não corresponde a nenhum registro ativo retorna `"Row not found for delete"` em vez de um erro genérico.

Formato de erros

Toda falha — de validação, de execução, ou de acesso negado — segue o mesmo formato, compatível com o padrão GraphQL:

```
{"errors":[{"message":"descrição do problema"}]}
```

Múltiplos problemas do mesmo request (por exemplo, vários campos inválidos numa mutation) podem vir juntos na mesma lista `errors`.

Erros comuns e o que significam

Mensagem	Causa
<code>Unknown or restricted table: '<nome>'</code>	Tabela não existe, está bloqueada, ou você esqueceu a palavra <code>mutation</code> numa chamada de escrita
<code>Unknown or restricted mutation: <nome></code>	Tabela existe para leitura, mas não está liberada em <code>allowMutations</code>
<code>Mutation '<nome>' requires an 'input' argument</code>	A chamada não incluiu o argumento <code>input</code>
<code>Mutation '<nome>' argument 'input' must be an object</code>	<code>input</code> foi enviado como algo que não é um objeto (ex. um número ou string solto)
<code>Field '<campo>' is required</code>	Campo obrigatório (SX3) ausente no <code>input</code> de um <code>create</code>
<code>Field '<campo>' must be a string / must be numeric / must be boolean</code>	Tipo do valor enviado não bate com o tipo real do campo
<code>Field '<campo>' exceeds max length <N></code>	Valor de texto maior que o tamanho máximo do campo (<code>X3_TAMANHO</code>)
<code>Missing key field '<campo>' for update / for delete</code>	Faltou um campo de chave no <code>input</code> de <code>update</code> / <code>delete</code>
<code>Update requires at least one non-key field</code>	O <code>input</code> de um <code>update</code> só trouxe campos de chave, nada para alterar
<code>Row not found for update / for delete</code>	A chave enviada não corresponde a nenhum registro ativo
<code>Write failed for '<mutation>'</code>	A escrita no banco falhou (SQL malformado, colisão de chave, etc.)

Coisas que a API intencionalmente não faz (ainda)

- **Não gera chave automaticamente:** você sempre informa o valor de `A1_COD` (ou equivalente) no `create` — não há numeração automática via SX5.
- **Não roda validações de negócio do Protheus:** apenas obrigatoriedade, tipo e tamanho (do SX3) são verificados. Regras de rotina, triggers e fórmulas de validação do cadastro padrão não são executadas.
- **Não faz mutations em lote:** cada chamada de `create` / `update` / `delete` afeta um único registro.
- **Não tem autenticação por usuário ainda:** o controle de acesso atual é só por configuração de servidor (tabelas/campos bloqueados). Isso está previsto para um sub-projeto futuro (Auth).

Veja `manual-implementacao.md` para as limitações operacionais (concorrência em `create`, dependência do índice SIX) que quem consome a API deve conhecer antes de depender dela em produção.